

Ajax e la programmazione asincrona

Rev 6.5 del 28/01/2026

Introduzione ai Web Services	2
Elementi base di Ajax	2
L'oggetto XMLHttpRequest	3
L'oggetto Promise	6
Oggetti per l'invio di una richiesta Ajax	7
\$.ajax	7
axios.js	9
fetch	11
 Creazione di una Promise	14
async - await	16
Promise.all()	18
JSON Server	20
Note sulle HTTP Request	23
 Il controllo input[type=file]	24
Upload di un file binario	25
 Scambio dati in formato XML	27
Cenni sulla libreria sweetAlert	28
Cenni sulla libreria chart.js	31
Cenni sulla libreria crypto-js	33

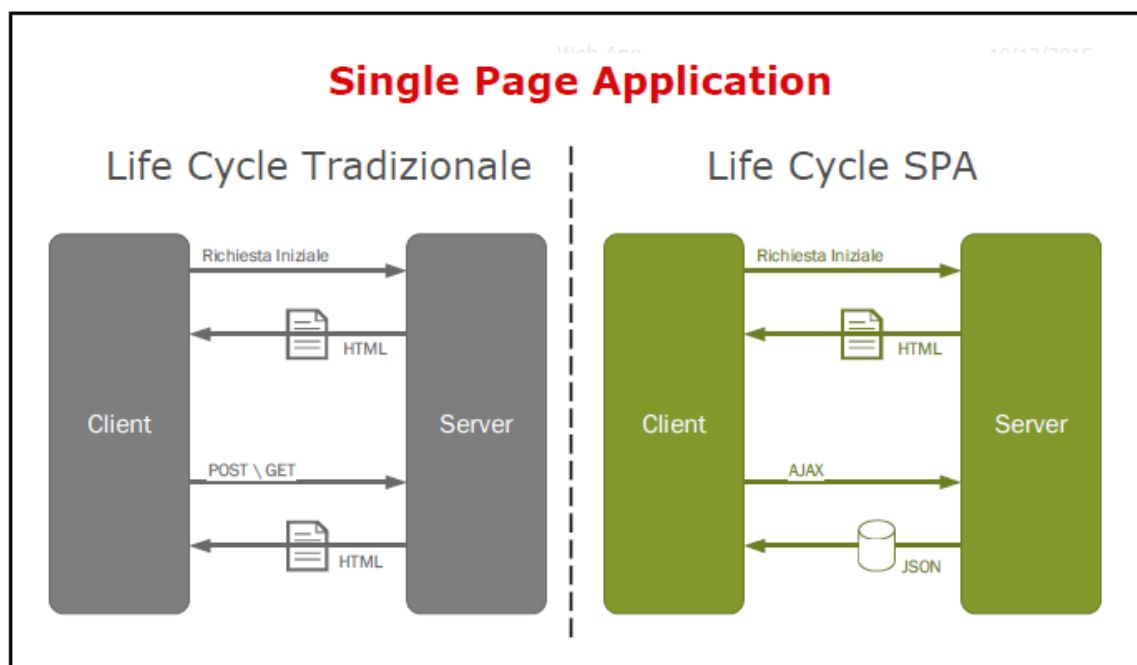
Introduzione ai Web Services

Un **web service** (detto anche API = Application Programming Interface), *da non confondere con web server*, è una **applicazione** in esecuzione su un web server che, a fronte di una richiesta http, restituisce un insieme di **dati** (in formato XML o JSON).

Il concetto di **web service** nasce negli anni 2004-2005 quando ci si rese conto che ricaricare ogni volta una intera pagina HTML risultava molto oneroso, mentre sarebbe stato molto più semplice e conveniente ricaricare di volta in volta soltanto i dati necessari da visualizzare. In questo modo le applicazioni risultano molto più veloci e i dati scambiati fra client e server si riducono notevolmente.

AJAX è la tecnologia utilizzata da un client per richiedere i dati esposti da un Web Service.

Insieme ad Ajax nasce anche il concetto di SPA **Single Page Application**. Il client effettua una unica richiesta iniziale di pagina e poi provvede ad aggiornare SOLO i dati tramite richieste ajax successive.



Elementi Base di Ajax

Asynchronous JavaScript And XML

javascript perché nasce con javascript in ambito web con lo scopo di consentire l'aggiornamento dinamico dei dati contenuti in una pagina html senza dover ricaricare l'intera pagina. Dopo di che Ajax è stato esteso a quasi tutti gli ambienti di programmazione (ad es nelle applicazioni desktop scritte in C#, nelle app per smartphone scritte in Android o IOS, etc.)

xml perché inizialmente i web service restituivano i dati in formato XML. Oggi ajax può restituire dati in qualsiasi formato (XML, JSON, CSV o anche semplice testo)

asynchronous perché si tratta di una tecnologia **asincrona** nel senso che la richiesta viene inviata in background all'interno di un 'thread' separato senza interferire con il comportamento della pagina. L'utente, una volta inviata la richiesta, può continuare ad interagire con l'interfaccia grafica anche mentre l'applicazione è in attesa dei dati.

Concetto contrapposto al modello tradizionale di **comunicazione sincrona** tipica delle vecchie applicazioni web (le cosiddette applicazioni Web Form) in cui, in corrispondenza del submit, viene inviata una richiesta al server rimanendo poi in attesa del caricamento della nuova pagina.

Per inviare le richieste ajax **non** si utilizza il pulsante di submit, ma un **normale button html** che richiama una procedura javascript, la quale :

1. **invia la richiesta**
2. **si mette in ascolto di un evento che scatterà in corrispondenza dell'arrivo dei dati**
3. **visualizza i dati all'interno della pagina.**

Tipi di chiamate http

Per inviare una richiesta ajax ad un web service sono disponibili principalmente due tipi di chiamate :

- **http GET**
- **http POST**

In entrambi i casi occorre 'passare' al web service alcune informazioni riguardo ai dati che si vogliono richiedere. Ad esempio 'mandami tutte le informazioni dello studente Pippo'. Pippo dovrà essere passato come **parametro** al web service. La differenza sostanziale tra i metodi GET e POST consiste nel come vengono inviati i parametri al web service:

- nel caso delle chiamate **GET** i parametri vengono trasmessi al server in modo visibile **in formato URL-encoded** in coda alla URL Es: <http://miosito.it/studenti?nome=Pippo&classe=1C>
Un **?** suddivide la URL dai parametri
Un **&** suddivide i vari parametri fra loro
- nel caso delle chiamate **POST** i parametri vengono trasmessi al server in forma nascosta all'interno del body della http request (che nel caso delle chiamate GET rimane vuoto)
Nelle chiamate POST i parametri possono essere trasmessi in qualsiasi formato.
Il più usato è il formato **JSON** trasmesso in forma serializzata.

Tipologie di web services

I web service basati direttamente sui metodi http GET e http POST e sul concetto di **risorsa** sono detti

- **REST** *REpresentational State Transfer* basati sul concetto di collegamento fra **risorse**

Esisteva in passato un'altra tipologia di web service oggi sempre meno diffusi detti

- **SOAP** *Simple Object Access Protocol* basati sul concetto di **servizio**

L'oggetto XMLHttpRequest

In javascript l'oggetto base per l'invio di una richiesta Ajax ad un web server è l'oggetto **XMLHttpRequest** il cui primo standard ufficiale è stato rilasciato dal World Wide Web Consortium (**W3C**) il 5 aprile 2006.

Un semplice esempio di utilizzo di Ajax potrebbe essere la scelta di un nuovo nickname in fase di creazione di un nuovo account su un sito web. In corrispondenza di ogni carattere digitato si invia una richiesta al server chiedendo se il nickname fino a quel momento inserito è valido oppure no.

A tale scopo può essere utilizzato l'evento onKeyUp della Text Box.

- in corrispondenza di ogni carattere, java Script invia in tempo reale una richiesta al server
- Il server valuta se il nome fin'ora inserito è valido o no elaborando una risposta molto 'leggera'.
- In base alla risposta Java Script aggiorna opportunamente l'aspetto del textbox

Invio della richiesta

```
var richiesta = new XMLHttpRequest();  
var url="controlla.php?parametro=" + encodeURIComponent(txtUsername.value);  
  
// apro la connessione TCP con il server  
richiesta.open("GET", url, true);  
  
// le requestHeader possono essere assegnate solo DOPO l'apertura della connessione  
richiesta.setRequestHeader(  
    "Content-type", "application/x-www-form-urlencoded; charset=utf-8");  
  
// funzione di callback da eseguire in corrispondenza della risposta  
richiesta.onreadystatechange = aggiorna;  
richiesta.send(null);
```

Parametri del metodo open

1. Il metodo con cui inviare i parametri al server (GET / POST)
2. La url della risorsa richiesta. La url può essere espressa in due modi:
 - path relativo a partire dalla cartella corrente. Es `url="controlla.php"`
 - path assoluto a partire dalla cartellahtdocs. Es `url="/5B/ese12/controlla.php"`La funzione `encodeURIComponent` consente di codificare eventuali caratteri speciali.
3. La modalità di esecuzione della send (true=asincrona, false=sincrona). Se si intende gestire una funzione di callback per la lettura della risposta, l'invio dovrà necessariamente essere asincrono.

Attributi da associare alla richiesta

- L'attributo `setRequestHeader` definisce il formato dei parametri da inviare al server.
- L'attributo `onreadystatechange` consente di definire un riferimento alla funzione JavaScript di callback che dovrà essere eseguita in corrispondenza del ricevimento della risposta.
- La funzione di callback **NON è obbligatoria. Il server (nel caso ad es di comandi DML) può anche NON inviare una risposta, nel qual caso la funzione di callback deve essere omessa.**

Il metodo send()

- Il metodo `send` consente di inviare la richiesta al server. Il parametro del metodo send vale:
 - **null** nel caso di richieste di tipo GET (come quella attuale)
 - nel caso delle richieste POST contiene l'elenco dei parametri in formato nome=valore scritti all'interno di una unica stringa e separati da & (oppure scritti in formato json o formData).

Poiché il client potrebbe inviare una nuova richiesta prima che sia giunta la risposta alla richiesta precedente, è buona regola **istanziare** un apposito oggetto `XMLHttpRequest` per ogni comunicazione, oppure inviare la nuova richiesta soltanto in corrispondenza del ricevimento della risposta precedente.

Gestione della risposta

La risposta testuale elaborata dal server viene restituita all'interno della proprietà `.responseText` dell'oggetto richiesta.

```
function aggiorna() {  
    if (richiesta.readyState==4 && richiesta.status==200)  
        alert(richiesta.responseText);  
}
```

La funzione `aggiorna()` può essere richiamata più volte nel corso della comunicazione. In corrispondenza delle varie chiamate il parametro `readyState` può assumere valori differenti :

```
0: request not initialized  
1: server connection established  
2: request received  
3: processing request  
4: request finished and response is ready
```

Se la risposta è pronta (`readyState=4`) e lo stato è corretto, allora si può leggere il contenuto della risposta.

Aggiornamento della pagina

Supponendo di usare un servizio lato server per il controllo di uno username e supponendo che tale servizio risponda :

- "OK" in caso di username valido
- "NOK" in caso di username non valido

lato client si può scrivere la seguente procedura di visualizzazione:

```
function aggiorna () {  
    if (richiesta.readyState==4 && richiesta.status==200) {  
        var msg = $("#msg");  
        var btn = $("#btnInvia");  
        var risposta = richiesta.responseText;  
  
        if (risposta.toUpperCase() == "OK") {  
            msg.text("Nome valido");  
            msg.css("color", "green");  
            btn.prop("disabled", false);  
        }  
        else if (risposta.toUpperCase() == "NOK"){  
            msg.text("Nome già esistente");  
            msg.css("color", "red");  
            btn.prop("disabled", true);  
        }  
        else  
            alert("Risposta non valida \n"+risposta);  
    }  
}
```

La programmazione asincrona : l'oggetto Promise

Si tratta di un oggetto mirato a facilitare la gestione dei processi asincroni. Tutti gli oggetti moderni utilizzati per l'invio di richieste ajax restituiscono al chiamante una **Promise**.

Con il termine **programmazione sincrona** si intende quella programmazione tradizionale in cui ogni operazione, prima di essere eseguita, attende la terminazione dell'operazione precedente.

La **programmazione asincrona** interviene nel momento in cui si eseguono operazioni particolarmente pesanti che altrimenti bloccherebbero l'interfaccia grafica; ad esempio:

- l'elaborazione grafica di una grande immagine
- l'attesa di ricezione dati da un server esterno
- la lettura di un file di grandi dimensioni

In questi casi l'applicazione lancia il comando di esecuzione e prosegue nell'esecuzione delle istruzioni successive senza fermarsi ad aspettare il risultato.

L'oggetto **Promise** serve appunto per gestire queste situazioni.

E' un oggetto che contiene un valore che non è ancora disponibile, ma che lo sarà in futuro.

Cioè ancora è una scatola che contiene un valore futuro.

Quando il valore arriva, la Promise lo consegna facendo scattare uno specifico evento, l'evento **then**

In caso di errori farà scattare un altro specifico evento, l'evento **catch**

Al termine del then e/o del catch viene sempre generato l'evento **finally**

Il main dovrà semplicemente gestire gli eventi **.then()** e **.catch()**, in particolare il then che scatterà non appena i dati saranno disponibili.

```
let promise = . . . . .
promise.then(function(data) {
    console.log(data)
})
promise.catch(function(err) {
    console.log(err.message)
})
promise.finally(function(){
    // close connection
})
```

Attenzione che all'interno del **then** **non si può eseguire un return**.

Se lo si esegue questo semplicemente va perso, oppure viene intercettato da un eventuale then posizionato in cascata al precedente, ma comunque non viene mai restituito al chiamante.

Oggetti per l'invio di richieste ajax

Per l'invio delle richieste ajax esistono diversi oggetti **ciascuno dei quali** restituisce al chiamante **SEMPRE** una **promise** che il chiamante potrà gestire all'interno del proprio codice.

\$.ajax()

E' un metodo statico della libreria jQuery. (non presente nella libreria jquery **slim**) che rappresenta un wrapper dell'oggetto java script **XMLHttpRequest** per l'invio di una richiesta ajax ad un server. Molto più semplice e flessibile rispetto ad XMLHttpRequest.

A differenza di **XMLHttpRequest**, \$.ajax() è in grado di gestire **richieste multiple** verso una stessa risorsa con una stessa funzione di callback. In pratica passa un indice al server che lo rimanderà indietro in modo che la callback possa capire a quale elemento è riferita la risposta.

jQuery però gestisce le promise in modo leggermente diverso rispetto a javascript. Senza entrare nel dettaglio del funzionamento delle promise jQuery si può semplificare dicendo che le promise jQuery :

- Non generano gli eventi **then** e **catch**
- Al loro posto generano gli eventi **done** e **fail** simili ma non identici rispetto a quelli javascript

Sintassi di \$.ajax()

\$.ajax() si aspetta come parametro un **json** di opzioni costituito dai seguenti campi:

```
let options={
  url: "/url?nome=pippo",
  type: "GET",           // default
  data: { "nome": "pippo" },
  contentType: "application/x-www-form-urlencoded; charset=UTF-8", // default
  dataType: "json",     // default
  timeout : 5000,
  async : true,         // default
  success: function(data, [textStatus], [jqXHR]) {
    console.log(data)
  },
  error : function(jqXHR, textStatus, str_error){
    if(jqXHR.status==0)
      alert("connection refused or server timeout");
    else if (jqXHR.status == 200)
      alert("Errore Formattazione dati\n" + jqXHR.responseText);
    else
      alert("Server Error: "+jqXHR.status+ " - " +jqXHR.responseText);
  },
  username: "nome utente se richiesto dal server",
  password: "password se richiesta dal server",
}
```

Richiamo

\$.ajax(options)

In caso di successo della richiesta i dati ricevuti dal server vengono iniettati alla funzione **success** che potrà provvedere alla loro elaborazioni.

Elaborare i dati all'interno delle `options` a livello operativo è una soluzione molto poco modulare (e abbastanza scomoda). Come detto però `$.ajax` restituisce una promise che potrà essere gestita direttamente all'interno del codice del chiamante.

L'attributo `contentType`

Indica il formato con cui passare i parametri al server. Può assumere i seguenti valori:

```
contentType: "text/plain; charset=utf-8"
contentType: "application/x-www-form-urlencoded; charset=utf-8"
contentType: "application/json; charset=utf-8"    //stringa json
contentType: "application/xml; charset=utf-8"    //stringa xml
```

Nel caso di chiamate GET eventuali parametri possono essere indifferentemente

- accodati alla url *oppure*
- inseriti come json all'interno del campo **data** delle Options

Se `contentType: "application/x-www-form-urlencoded"`, i parametri passati a `$.ajax()` in formato JSON, **vengono automaticamente a convertirli in formato urlencoded**. Esempio:

```
let json = {"user1":{"nome":"pippo", "hobbies":["calcio","pesca"]}}
?user1[nome]=pippo&user1[hobbies][0]=calcio&user1[hobbies][1]=pesca
```

I parametri **POST** possono invece essere passati direttamente in un formato **"application/json"**. In questo caso, prima di inviare la richiesta, occorre **SERIALIZZARE** l'intero json.

L'attributo `dataType`

Indica il formato con cui `$.ajax()` deve restituire al chiamante i dati ricevuti dal server.

Può assumere i seguenti valori esprimibili **solamente** in modo diretto senza `application/` davanti.

```
dataType: "text"
dataType: "json"
dataType: "xml"
dataType: "html"
dataType: "script"
```

- Scegliendo `"text"` la risposta viene restituita a `onSuccess()` cos'è com'è, indipendentemente dal fatto che sia testo oppure json oppure xml.
- Scegliendo **json**, `$.ajax` provvede automaticamente ad eseguire il **parsing** dello stream json ricevuto, restituendo a `onSuccess()` un oggetto json. Idem per xml

Il metodo `success`(data, textStatus, jqXHR)

Viene richiamato in corrispondenza della ricezione della risposta. In caso di content-type non testuale (ad esempio json o xml) **provvede automaticamente a parsificare la risposta ricevuta restituendo al chiamante l'object corrispondente**. Questa conversione automatica viene fatta in un "thread" separato e quindi alleggerisce la nostra applicazione..

`textStatus` indica lo stato in cui si è conclusa la XMLHttpRequest

`jqXHR` è un riferimento all'oggetto XMLHttpRequest sottostante utilizzato per inviare la richiesta

Il metodo `error`(jqXHR, textStatus, str_error)

In caso di errore, invece di richiamare la funzione **success**, viene richiamata la funzione **error**. Questa funzione viene richiamata :

- **in caso di timeout**
- in corrispondenza della ricezione di un codice di **errore diverso da 200**
- in caso di ricezione di status==200 ma con un **oggetto json non valido** (se **dataType="json"**) (ad esempio se il server va in syntax error e restituisce un messaggio di errore)

jqXHR è un riferimento all'oggetto XMLHttpRequest sottostante utilizzato per inviare la richiesta

testStatus indica lo stato in cui si è conclusa la XMLHttpRequest

Il terzo parametro rappresenta un msg di errore fisso legato al codice di errore

L'attributo async

Impostando il valore **false** il metodo diventa sincrono, bloccando di fatto l'interfaccia grafica fino a quando non sono arrivati i dati

Scrittura di una funzione compatta per l'invio delle richieste ajax

```
function inviaRichiesta(method, url, parameters={}) {  
  const URL = "https://randomuser.me"  
  let contentType;  
  if (method.toUpperCase()=="GET")  
    contentType="application/x-www-form-urlencoded;charset=utf-8";  
  else{  
    contentType = "application/json; charset=utf-8"  
    parameters = JSON.stringify(parameters);  
  }  
  let options={  
    url: URL + url,  
    type: method  
    data: parameters,  
    contentType: contentType  
    dataType: "json", // default  
    timeout : 5000,  
  }  
  return $.ajax(options)  
}  
function errore (jqXHR, test_status, str_error) {stessa di prima}
```

Chiamante

```
let promise = inviaRichiesta("get", "/api?student=pippo") // oppure  
let promise = inviaRichiesta("get", "/api", {"student":"pippo"})  
promise.fail(errore);  
promise.done(function(data, test_status, jqXHR){  
  console.log(data); });
```

axios.js

Visto l'attuale declino di jQuery, **axios.js** (dal greco "**degnò**") rappresenta una degna alternativa a \$.ajax in direzione "vanilla" javascript. Restituisce una promise js con gli eventi .then e .catch. Dal sito ufficiale:

Axios is a promise-based HTTP Client for **node.js** and the **browser**. It is **isomorphic** (= it can run in the browser and node.js with the same codebase). On the server-side it uses the native node.js http module, while on the client (browser) it uses **XMLHttpRequest**.

La si può scaricare in versione **UMD** aprendola su un CDN e poi facendo "Salva pagina con nome".

Analogamente a \$.ajax() i parametri GET possono essere :

- accodati direttamente alla url in formato urlencoded oppure
- inseriti in formato **json** all'interno del campo **params** delle Options. **Axios provvederà automaticamente e convertire questi parametri in url-encoded e ad accodarli alla URL.**

I parametri POST possono essere passati in formato **json** all'interno del campo **data** delle options. La serializzazione viene fatta automaticamente da axios.

```
function inviaRichiesta(method, url, parameters={}) {
  let options = {
    "baseUrl": "http://webserver:3000", // indirizzo del web server
    "url": url, // risorsa da richiedere
    "method": method.toUpperCase() // GET or POST
    "headers": {"Accept": "application/json"},
    "responseType": "json", // tipo di risposta atteso
    "timeout": 5000 // tempo di attesa della risposta
  }
  if(method.toUpperCase() == "GET"){
    options.headers["Content-Type"]='application/x-www-form-urlencoded;charset=utf-8'
    options["params"]=parameters // plain object or URLSearchParams object
  }
  else{
    options.headers["Content-Type"] = 'application/json; charset=utf-8'
    options["data"]=parameters // Accept FormData, File, Blob
  }
  return axios(options) // return a promise
}

function errore(err) {
  if(!err.response)
    alert("Connection Refused or Server timeout");
  else if (err.response.status == 200)
    alert("Formato dei dati non corretto : " + err.response.data);
  else
    alert("ServerError: " + err.response.status + "-" + err.response.data)
}
```

Chiamante

Al **then** viene iniettata l'intera **http-response**. I dati ricevuti si trovano dentro **response.data**

```
let promise = inviaRichiesta("get", "/api?student=pippo") // oppure
let promise = inviaRichiesta("get", "/api", {"student":"pippo"})
promise.catch(errore) ;
promise.then(function(httpResponse) {
  console.log(data);
});
```

Nota sui parametri GET in formato url-encoded

Nel caso delle chiamate **GET** config["params"] accetta al suo interno anche dei **plain object**, cioè dei json semplici, cioè NON annidati. Tenere presente che tra i **plain object** rientrano anche i vettori enumerativi. Quindi è possibile passare come parametro sia un vettore enumerativo sia un json di 1° livello

```
{ "data" : [1,2,3,4] }
  viene tradotto in URL-encoded nel modo seguente:
  ?data[]=1&data[]=2& data[]=3& data[]=4

{ "data" : {"a":1, "b":2 } }
  viene tradotto in URL-encoded nel modo seguente:
  ?data[a]=1&data[b]=2
```

fetch

Metodo nativo javascript (2015) sostitutivo del vecchio XMLHttpRequest.

Fetch is the modern replacement for XMLHttpRequest: unlike XMLHttpRequest, which uses callbacks, Fetch is promise-based and is integrated with features of the modern web such as service workers and Cross-Origin Resource Sharing (CORS).

fetch() presenta un unico argomento obbligatorio, la URL della risorsa a cui accedere. E' possibile passare secondo parametro di **options**, che sono più o meno le stesse di \$.ajax() e axios().

La chiamata va sempre a buon fine, cioè fetch restituisce sempre un oggetto **response** indipendentemente dall'http status code della risposta (sia 200 oppure un qualunque codice di errore). fetch genera errore soltanto in caso di errore di rete a seguito di timeout.

L' oggetto **Response** restituito da fetch espone:

- una proprietà **ok** impostata su **true** in caso di ok o su **false** per risposte diverse da 2xx

Espone inoltre

- una proprietà **status** contenente l'http response code
- una proprietà **statusText** contenente un http response message fisso

L'oggetto **Response** non contiene però direttamente l'effettivo corpo della risposta JSON (come nel caso di axios), ma per ottenere i dati veri e propri occorre richiamare un ulteriore metodo asincrono **response.json()** il quale restituisce una seconda **promise** che si risolve iniettando i dati veri e propri.

Esempio completo di invio di una richiesta

```
const _URL = "" // "http://localhost:1337"

async function inviaRichiesta(method, url="", params={}) {
  method = method.toUpperCase()
  let options = {
    "method": method,
    "headers": {},
    "mode": "cors", // default
    "cache": "no-cache", // default
    "credentials": "same-origin", // default
    "redirect": "follow", // default
    "referrerPolicy": "no-referrer", // default no-referrer-when-downgrade
    // riduce il timeout rispetto al default (6s) Non sembra possibile increm
    // "signal": AbortSignal.timeout(500)
  }

  if (method=="GET") {
    const queryParams = new URLSearchParams();
    for (let key in params) {
      let value = params[key];
      if (value && typeof value === "object")
        queryParams.append(key, JSON.stringify(value));
      else
        queryParams.append(key, value);
    }
    url += "?" + queryParams.toString()
    options.headers["Content-Type"]="application/x-www-form-urlencoded"
  }
}
```

```
else if(params instanceof FormData)
    options["body"]=params // Accept FormData, File, Blob
else{
    options.headers["Content-Type"]="application/json";
    options["body"] = JSON.stringify(params) }
}

try {
    const response = await fetch(_URL + url, options)
    // Leggo il body come semplice testo
    const text = await response.text()
    // Se lo status HTTP non è ok
    if (!response.ok) {
        return { status: response.status, err: text }
    }
    // Provo a fare il parse JSON
    let data = null
    if (text) {
        try { data = JSON.parse(text) }
        catch (err) {
            return { status: 422, err: "Response contains invalid JSON" }
        }
    }
    // Success
    return { status: response.status, data }
}
catch (err) {
    return { status: 408, err: "Connection refused or server timeout" }
}
}
```

Chiamante

```
let response = inviaRichiesta("POST", "https://example.com/answer", {n:42} )
```

Note:

- 1) Notare il `.toUpperCase()` sul method. `fetch()` esige che i metodi siano scritti in **MAIUSCOLO** !
- 2) `fetch` non espone i classici eventi per controllare lo stato di avanzamento dell'upload
- 3) CORS requests may only use the HTTP or HTTPS **URL scheme**.
`fetch` non invia richieste con **URL schema** non `http`, per cui non è utilizzabile con il protocollo `file://`
- 4) Eventuali **parametri GET** devono essere manualmente accodati alla URL. Non esiste il `config["params"]` di `axios` che esegue la conversione URL-encoded in automatico.
- 5) Occhio alle **headers**, che vanno sempre scritte fra parentesi quadre. Non esiste la versione camelCase
- 6) Notare che **statusText** non contiene il messaggio di errore impostato dal server, ma un testo fisso di errore http associato al codice di errore ricevuto. Per accedere al messaggio ricevuto dal server occorre utilizzare il metodo asincrono **response.text()** come nell'esempio precedente.
- 7) Confronto Axios – Fetch : <https://blog.logrocket.com/axios-vs-fetch-best-http-requests/>

Metodi dell'oggetto response

Esiste un apposito metodo **asincrono** per ogni possibile oggetto ammesso all'interno del campo **body**:

- `.text()` : Returns the response as string data
- `.json()` : Returns the response as JSON
- `.formData()` : Return the response as `FormData` object (which stores key-value pairs of string data)
- `.blob()` : Returns the response as blob object (binary data along with its encoding)
- `.arrayBuffer()` : Return the response as `ArrayBuffer` (low-level representation of binary data)

Upload di un file binario

Come detto fetch accetta un Content-Type= multipart/form-data senza alcun ulteriore vincolo. Per l'upload dei file binari, la tendenza attuale è quella di utilizzare il metodo PUT

```
const formData = new FormData();
formData.append("username", "abc123");
formData.append("avatar", txtFile.files[0]);
fetch("https://example.com/profile/avatar", {
  method: "PUT",
  body: formData,
})
```

Interceptors

```
const requestInterceptor = function(request) {
  if (!request.headers)
    request.headers = new Headers();
  if ("authToken" in localStorage) {
    let authToken = localStorage.getItem("authToken");
    request.headers.set('authorization', authToken);
  }
  return request;
}

const responseInterceptor = function(response) {
  if (response.headers) {
    let authToken = response.headers.get("authorization")
    if(authToken)
      localStorage.setItem("authToken", authToken)
  }
}
```

Approfondimenti sulle fetchOptions

mode : **cors** = normale funzionamento CORS

no-cors = cioè senza resource sharing. Consente solo SAFE request

same-origin = . Consente solo chiamate sulla stessa origine

credentials: **omit** = non invia credenziali (cookies e HTTP Credential)

same-origin = invia le credenziali solo nelle same-origin request

include = invia le credenziali sia nelle same-origin request che nelle cross-origin
(il valore **include** è incompatibile con Access-Control-Allow-Origin = "*")

Content-Type: Può assumere i valori application/x-www-form-urlencoded,
application/json, text/plain, multipart/form-data,

body : può contenere i seguenti oggetti: String (or String Literal), URLSearchParams,
FormData, File, Blob, ArrayBuffer, TypedArray, DataView

referrerPolicy: L'intestazione http **Referer** è simile a **Origin** ma, mentre Origin contiene SOLO l'origine della richiesta (protocollo, dominio e porta), Referer contiene l'intero path della richiesta compresi i parametri. Al fine di preservare la privacy, le policy del Referer della richiesta possono definire i dati che possono essere inclusi / omessi. E' possibile anche omettere l'intera l'intestazione del Referer. Viceversa Origin non può essere esclusa

"no-referrer" – non invia mai il Referer.

"unsafe-url" – invia sempre l'url completo nel Referer, anche per richieste HTTPS→HTTP.

Purtroppo tra le **fetchOptions** no c'è un **timeout** che deve eventualmente essere gestito a mano.

Creazione di una Promise

Si supponga di avere una procedura asincrona `elaboraImmagine()` molto pesante in termini di esecuzione e n. di righe di codice. La riga `imgLibrary.elabora(img)` lancia l'elaborazione dell'immagine e poi termina immediatamente. Quando l'immagine sarà stata elaborata verrà generato un evento `onEnd` a cui viene iniettata l'immagine elaborata che dovrà essere utilizzata dal main.

```
function elaboraImmagine(img) {  
    imgLibrary.elabora(img)  
    imgLibrary.onEnd(finalImage) { // elaborazione terminata  
        console.log(finalImage)  
        return finalImage !!!  
    }  
}
```

Questa `finalImage` NON può essere ritornata al main; dal momento che **una procedura di evento non può eseguire un return !** Per cui il codice di gestione della `finalImage` dovrebbe essere scritto all'interno della callback finale. Un eventuale return potrebbe essere eseguito soltanto ALLA FINE della procedura `elaboraImmagine()`, dove però la variabile `finalImage` non è ancora disponibile.

A livello di leggibilità (ma anche di programmazione) sarebbe molto più comodo se `elaboraImmagine` fosse sincrona, cioè bloccante fino al termine dell'elaborazione e restituendo il risultato al main:

```
finalImage = elaboraImmagine(img)
```

Nella programmazione web questo però non è possibile perché bloccherebbe l'interfaccia grafica per tutto il tempo di elaborazione.

Creazione di una Promise

Per poter utilizzare la `finalImage` al di fuori dell'evento `onEnd`, l'intero codice della funzione `elaboraImmagine` deve essere 'avvolto' all'interno di un oggetto **Promise** il quale, al termine dell'elaborazione, farà scattare il `.then`

A tal fine occorre istanziare una `Promise()` che ha come unico parametro una function a cui vengono automaticamente iniettati due puntatori a funzione: **resolve** e **reject**.

- Nel momento in cui `elaboraImmagine()` richiama il metodo **resolve**, automaticamente la promise provvede a generare l'evento **then()**, iniettandogli il risultato passato a resolve. Tutti i parametri passati a `resolve()` vengono iniettati al `then()`
- Nel momento in cui `elaboraImmagine()` richiama il metodo **reject**, automaticamente la promise provvede a generare un evento **catch()**, iniettandogli un oggetto err relativo all'errore verificatosi

```
function elaboraImmagine (img) {  
    let promise = new Promise(function(resolve, reject) {  
        imgLibrary.elabora(img)  
        imgLibrary.onEnd(err, finalImage) { // elaborazione terminata  
            if(err)  
                reject(err)  
            else  
                resolve(finalImage);  
        }  
    })  
    return promise;  
}
```

Cioè la funzione `elaboraImmagine()` istanzia la Promise e termina immediatamente restituendo la Promise al main. Il main dovrà mettersi in ascolto della promise in attesa che 'scatti' il `then` o il `catch`.

main

```
let promise = elaboraImmagine(img)
promise.then(function(finalImage) { visualizza(finalImage) } )
promise.catch(function(err) { alert(err) } )
```

Se il codice interno non richiama né il metodo **resolve** né il metodo **reject**, la Promise rimane “appesa”, cioè non vengono generati gli eventi **then()**, **catch()** e tantomeno **finally()**

Altro Esempio

Inclusione di un `setTimeout` all'interno di una promise :

```
function timer() {
    let promise = new Promise(function (resolve, reject) {
        setTimeout(function () { resolve("sono trascorsi 1000 ms") }, 1000);
    })
    return promise
}

let attendiUnSecondo = timer() // attendiUnSecondo sarà una Promise
attendiUnSecondo.then(function(msg) {
    console.log(msg) // sono trascorsi 1000 ms
})
```

Concatenamento degli eventi

Il codice precedente può anche essere scritto nel modo seguente, senza l'utilizzo della variabile intermedia `let promise =`, concatenando il `.then()` e `.catch()` direttamente alla funzione principale.

```
elaboraImmagine(img)
    .then(function(finalImage) {
        console.log(finalImage)
    })
    .catch(function(err) {
        console.log(err.message)
    })
```

Vantaggi

In questo modo, diventa possibile associare ad ogni promise più funzioni di callback in cascata in quanto ogni promise restituisce a sua volta un'altra promise

```
promise.then(function(){ console.log(1); })
    .then(function(){ console.log(2); })
```

Svantaggi

Questa sintassi non è del tutto equivalente alla precedente.

Sia `.then` che `.catch` restituiscono a loro volta una **nuova** promise che ‘maschera’ la promise principale alla quale possono essere associati un nuovo `.then()` ed un nuovo `.catch()` Per cui:

- Se si utilizza prima `.then` e dopo il `.catch`, il `catch` diventa gestore dell'errore sia rispetto alla promise principale, sia rispetto al codice interno del `then` e pertanto può visualizzare errori fuorvianti che ‘mascherano’ il vero motivo dell'errore.
- Se si mette prima il `.catch` e dopo il `.then`, se il `catch` non introduce altri errori, il `then` viene eseguito in cascata. Assolutamente intollerabile !

La soluzione migliore è quella di usare SEMPRE una variabile intermedia.

Nota Gli eventi `.then` e `.catch` vengono comunque eseguiti **anche** se la loro assegnazione viene fatta *dopo* che la promise è terminata (cioè *dopo* che sono stati invocati i metodi `resolve` o `reject`).

Il costrutto Async / Await

Rappresenta una modalità alternativa rispetto al `.then` e `.catch` per gestire una promise restituita da una funzione asincrona. Consente di far apparire una funzione asincrona come 'bloccante'.

"await" makes JavaScript wait until the promise settles, and then go on with the result. That doesn't cost any CPU resources, because the engine can do other jobs meanwhile: execute other scripts, handle events etc. It's just a more elegant syntax of getting the promise result than promise.then(), easier to read and write.

Si riconsideri l'esempio iniziale di una funzione asincrona che restituisce una promise contenente il risultato di una elaborazione grafica di una immagine:

```
function elaboraImmagine(img) {  
  let promise = new Promise(function(resolve, reject) {  
    _lastLibrary.onEnd(err, finalImage){  
      if(err)  
        reject(err)  
      else  
        resolve(finalImage);  
    })  
    return promise;  
  }  
}
```

Sintassi then / catch

```
let promise = elaboraImmagine(img)  
promise.then(function (finalImage){ })  
promise.catch( )
```

Sintassi async / await

```
async function visualizza(){  
  let finaleImage = await elaboraImmagine(img)
```

- **await** attende la risoluzione della promise e, **anziché restituire la promise, restituisce direttamente i dati che nella versione 'tradizionale' verrebbero iniettati al then.**
- la parola chiave **await** può essere utilizzata soltanto all'interno di una funzione dichiarata **async**.
- Una funzione dichiarata **async** **restituisce sempre una promise** che, a sua volta, deve essere richiamata con **await** oppure con il `then/catch`.

Return del risultato

La grande comodità dell'**await** sta nel fatto che i dati restituiti da **await** possono essere direttamente **ritornati** al chiamante. Attenzione però che, se una funzione dichiarata **async** ritorna un qualunque risultato, in realtà ciò che viene ritornato **NON sono i dati** ma **una nuova Promise** contenente i dati indicati dopo il `return`. In sostanza l'**await** rimanda la gestione della promise al chiamante il quale a sua volta dovrà gestire questa **Promise** con il `then` e `catch` oppure con un nuovo **await**.

Utilizzo con le arrow function

```
users.forEach(async (user) => { } )
```

Gestione dell'errore

Se si intende gestire anche l'errore si può concatenare il `.catch` direttamente in coda alla funzione.

```
let data = await elaboraImmagine(img).catch(function(err){ ..... })
```

Attenzione però che il catch non interrompe il flusso. Se il catch esegue una return, il valore ritornato dalla catch **viene salvato dentro data** e questo può creare confusione. Se il catch non esegue nessun return, viene automaticamente eseguito un return undefined, per cui **data** risulterà **undefined**.

Se invece fosse necessario eseguire un return all'interno del catch, la soluzione migliore è quella di utilizzare una soluzione più chiara e più semplice basata su un TRY – CATCH :

```
async function myFunction() {
  try { let data = await elaboraImmagine(img) return {"ok":true, data} }
  catch (err) { return {"ok":false, err} }
```

Approfondimenti su await

Notare che quando si scrive

```
let ris = await something;
```

something non deve essere necessariamente una Promise.

- Se è una Promise, await “attende” che la Promise venga risolta.
- Se NON è una Promise, await **converte implicitamente something in una Promise risolta**.
Di conseguenza, in questi casi, NON sarà possibile concatenare il catch

Ad esempio **Swal.fire()** che non restituisce una vera Promise javascript , ma un oggetto detto **thenable**, che await converte in una promise NON è possibile concatenare il catch.

await ovviamente può essere utilizzato anche direttamente davanti ad una **promise** dichiarata nella funzione stessa:

```
async function foo() {
  let promise = new Promise(function(resolve, reject) {
    setTimeout(() => resolve("done!"), 1000)
  });
  let result = await promise;
  alert(result); // "done!"
}
```

Approfondimenti su async

La parola chiave **async**, oltre che nel caso tipico degli esempi precedenti, può anche essere utilizzata davanti a qualsiasi funzione **per fare in modo che la funzione ritorni una promise**.

```
async function foo() {
  return 1;
}
```

Nel caso in cui il codice interno alla funzione ritorni una <non-promise>, automaticamente il risultato verrà “wrappato” all'interno di una promise. Nell'esempio la funzione restituirà una promise con dentro il valore 1

```
let promise = foo();
promise.then(function(val) {
  alert(val); // 1
// oppure, più semplicemente
promise.then(alert); // 1
```

In alternativa all'uso di **async**, la funzione foo() potrebbe anche essere scritta nel modo seguente, con un Promise.resolve() esplicito:

```
function foo() {return Promise.resolve(1)}
```

`$(document).ready`

La funzione jQuery `$(document).ready` non consente di dichiarare **async** la funzione di callback. Nel caso occorre utilizzare l'analoga javascript, cioè `window.onload=async function()`

Il metodo Promise.all()

Capita talvolta di dover eseguire parallelamente più operazioni asincrone.

Si supponga di voler prenotare più **ombrelloni** di un certo stabilimento balneare, in cui occorre eseguire una richiesta di prenotazione per ogni ID. Se tutte le prenotazioni sono andate a buon fine si vuole visualizzare un unico messaggio riassuntivo di **OK**. Gli approcci possibili sono due:

1. Eseguire le richieste in modo **annidato** ognuna all'interno del **then** della precedente. In corrispondenza della terminazione dell'ultima richiesta si visualizza l'**OK**. Pesante sia dal punto di vista della scrittura sia dal punto di vista dell'esecuzione.
2. Lanciare tutte le query in sequenza una dopo l'altra (con conseguente elaborazione parallela). Come faccio a dare il messaggio di OK? L'ordine di esecuzione delle richieste non è detto che coincida con l'ordine di lancio. Occorre pertanto gestire un contatore incrementato dopo ogni query. Quando il contatore raggiunge `vet.length` si dà l'OK. Soluzione efficace ma un po' spartana.

Un terzo approccio più strutturato, consiste nell'utilizzo del metodo **Promise.all()** il quale si aspetta come parametro un vettore di promises e genera una unica finalPromise in cui :

- il **.then()** viene generato se tutte le promises sono state risolte con esito positivo.
- Se anche **una sola promise** viene risolta con esito negativo, verrà generato il **.catch()** a cui viene iniettato l'errore generato dalla prima Promise che fallisce.

Per cui, quando si hanno più richieste da lanciare in parallelo, nel momento in cui ciascuna singola richiesta restituisce la propria promise, **si può salvare questa promise all'interno di un apposito vettore**.

Poi si mette il metodo Promise.all() in ascolto su questo vettore. Il metodo Promise.all() attende che tutte le promise del vettore vengano risolte e, al termine, genera una nuova Promise.

Al **.then()** della finalPromise viene iniettato un **vettore enumerativo** di json contenente tutte le **httpResponse** iniettate alle varie singole Promise, **nello stesso ordine con cui state lanciate le Promises**, indipendentemente dall'ordine di terminazione delle stesse. In questo modo si risolve perfettamente il problema dell'asincronicità delle chiamate. ESEMPIO:

```
let ombrelloniDaPrenotare = [7, 12, 24, 52]
let promises = []

for (let id of ombrelloniDaPrenotare)
  let request = inviaRichiesta("PATCH", `/ombrelloni/${id}` {stato:true})
  promises.push(request)
}

let finalPromise = Promise.all(promises)
finalPromise.then(function(httpResponses) {
  alert("Prenotazioni eseguite correttamente")
  for (httpResponse of httpResponses) console.log(httpResponse.data)
});
finalPromise.catch(function(err) {
  console.log(err)
})
```

Recentemente è stata aggiunta anche **Promise.allSettled()** che aspetta che tutte le promise siano terminate (con successo o con errore) e inietta al then un vettore con **tutti** i risultati (positivi e negativi).

L'array iniettato al then sarà un array di json così strutturato :

```
{status:"fulfilled", value:result} // per le risposte con successo
{status:"rejected", reason:error} // per gli errori.
```

```
httpResponses.forEach((httpResponse) => {
  if (httpResponse.status == "fulfilled") console.log(httpResponse.value.data)
  else if (httpResponse.status == "rejected") console.log(httpResponse.reason)
})
```

Promise.all() abbinato al metodo map()

Si consideri il seguente caso di studio.

Si parte da un vettore di risposte selezionate da uno studente durante un quiz a risposte chiuse in cui lo studente deve scegliere la risposta esatta mediante una sequenza di radio buttons.

```
let risposte = [
  {"idDomanda" : "1", "rispostaSelezionata" : 3},
  {"idDomanda" : "2", "rispostaSelezionata" : 5},
  {"idDomanda" : "3", "rispostaSelezionata" : 0}
]
```

Si vogliono confrontare le risposte selezionate con le risposte corrette contenute all'interno di un database.

```
let voto = 0;
const risultati = await Promise.all(
  risposte.map(async function(risposta) {
    try {
      const domanda = await tabellaDomande.findOne({id:risposta.idDomanda})
      if (domanda) {
        const correct = domanda.correct == risposta.rispostaSelezionata;
        if (correct)
          voto += 1;
        else
          voto -= 0.25;
        return { ...risposta, correct };
      }
    }
    catch (err) {
      console.log(err)
    }
    return risposta;
  })
)
```

Al metodo `risposte.map()` viene passata una **funzione asincrona** alla quale, ad ogni ciclo, viene iniettato un singolo json del vettore `risposte`.

La funzione va nel database a cercare la domanda avente come `id = risposta.idDomanda`

Se la domanda viene trovata si confronta `domanda.correct` con `risposta.rispostaSelezionata`.

Se la risposta data dallo studente è corretta si aggiunge 1 voto altrimenti si sottrae 0.25

Alla fine, per ogni risposta, si crea un nuovo json che contiene le stesse due chiavi del json originale `risposta`, con l'aggiunta di una chiave `correct` che potrà essere true/false.

In caso di errori (catch) oppure se la domanda non è stata trovata con `if(domanda)`, la callback del metodo `.map()` ritorna semplicemente la risposta originale senza la proprietà `correct`.

Questo return finale è abbastanza inutile.

Se non ci fosse verrebbe automaticamente eseguito un `return undefined`

Ricordando che quando una funzione asincrona esegue un return, ciò che in realtà viene ritornato non sono i **dati** ma una **promise**, possiamo concludere che il metodo `.map()` ritorna un vettore enumerativo di promise (una per ogni domanda analizzata) in cui ciascuna promise contiene il singolo json `risposta` contenente a sua volta anche il campo `correct`.

Questo vettore enumerativo di promise viene passato a `Promise.all()` il quale, quando tutte le promise saranno state risolte, restituirà un vettore **risultati** contenente per ogni cella **il singolo json ritornato dal metodo .map()**.

JSON-Server

JSON-Server è una piccola utility disponibile in ambiente nodejs in grado di rispondere a chiamate Ajax e restituire al chiamante il contenuto di un `file.json` memorizzato all'interno della cartella di lavoro.

Passi necessari all'installazione ed utilizzo di json-server

1. Installare NodeJS (<https://nodejs.org>)
2. Dal pulsante WINDOWS selezionare **Node / Nodejs Command Prompt**
Il prompt normale dal 2024 ha iniziato a segnalare problemi di sicurezza
3. Installare globalmente `json-server`
`npm install -g json-server`
4. Scrivere nella cartella di progetto un file.json contenente il database dell'esercizio
5. Aprire un **Nodejs Command Prompt** nella cartella di lavoro
6. Lanciare json-server in modo che utilizzi ad esempio il file db.json
`json-server --watch db.json`
L'opzione `--watch` indica che il file deve essere monitorato con continuità. Dalla versione 2024 questa opzione viene settata in automatico e dunque non è più necessaria
7. Il server risponde con un messaggio che conferma l'attivazione in localhost sulla porta **3000** e visualizza l'elenco delle risorse contenute all'interno del file db.json. Se la porta non viene impostata automaticamente la si può specificare nel comando di avvio
`json-server --port 3000 db.json`

All'interno del file db.json si possono inserire più gruppi di dati (detti **risorse**) che possono essere soltanto **vettori associativi** (json). Dalla versione 2024 i vettori enumerativi non sono più ammessi.

Per default json-server viene avviato soltanto sull'interfaccia `localhost` e dunque non sarà visibile sulla rete. Per avviarlo sull'interfaccia di rete occorre utilizzare l'opzione `--host`

```
json-server --host 10.0.1.2 --port 3000 --watch db.json
```

Esempio di file db.json contenente una unica risorsa denominata people

```
{ people: [  
  {  
    "id": "abc1",  
    "nome": "Alfio",  
    "genere": "m",  
    "classe": "4B"  
  },  
  {  
    "id": "abc2",  
    "nome": "Beatrice",  
    "genere": "f",  
    "classe": "4B"  
  },  
  {  
    "id": "abc3",  
    "nome": "Carlo",  
    "genere": "m",  
    "classe": "1C"  
  }  
]}
```

Gestione degli ID

Notare innanzitutto che, a differenza di mongo, il campo non si chiama `_id` ma **id**

id deve tassativamente essere dichiarato come **stringa** di qualunque lunghezza.

Poiché le chiamate inviano delle stringhe, in caso di **id** numerici possono fallire.

Se non è necessario ai fini del progetto, ID **può anche essere omissso**, però poi diventa complicato eseguire le chiamate ajax

Per i record creati **dinamicamente da codice** (e solo in questo caso), ID viene creato automaticamente come stringa su 4 caratteri

A questo punto si può aprire un browser e :

- richiedendo la url `http://localhost:3000`, il server risponde con una semplice pagina di benvenuto.
- Se invece si concatena alla url il nome di una risorsa di primo livello, il server risponde inviando l'intero contenuto associato a quella risorsa: `http://localhost:3000/people`. E' possibile richiedere **qualsiasi risorsa di primo livello** presente nel file db.json

Richieste GET

Sono quelle più usate e consentono di richiedere dei dati al server

GET /people senza parametri restituisce un vettore enumerativo con tutte le persone
GET /people?genere=m&classe=4B Esegue una **AND** e restituisce i record che soddisfano le condizioni
GET /people?id=2 Analogo al precedente. restituisce un vettore enumerativo lungo 1
GET /people/2 Vale **SOLO** per l'ID. restituisce un singolo record relativo alla persona con id=2

Richiesta di un singolo record sulla base dell'ID

L'ID può essere direttamente accodato alla risorsa dopo lo slash nel qual caso il server restituisce un singolo record relativo alla persona avente id=2 oppure può essere passato in formato url-encoded dopo il punto interrogativo, nel qual caso il server restituisce un vettore enumerativo lungo 1

Richiesta di più record sulla base di altri campi

I parametri diversi dall'ID vanno concatenati alla url dopo il ? e suddivisi tra loro mediante una &, utilizzando cioè quello che si chiama formato **url-encoded**.

Ad esempio Cerca tutti gli studenti di genere maschile della classe 4B

GET /people?genere=m&classe=4B. // Tutti i maschi della classe 4B

`inviaRichiesta()`, per come è stata scritta, accetta i campi di filtro anche come terzo parametro in formato JSON (molto comodo) e provvede automaticamente a **convertirli** in url-encoded concatenandoli alla url. Esempo: `inviaRichiesta("GET", "/people", {"genere":"m", "classe":"4B"})`

Altre modalità di richiesta

In tutte le chiamate diverse dalla GET, **ID può essere passato SOLO in coda alla risorsa !**

DELETE /people/2 elimina il record con id=2. Non ha parametri.

json-server consente di eliminare **UN SOLO RECORD** alla volta ed **ESCLUSIVAMENTE** sulla base dell'id

PATCH /people/2 {"residenza":"genola", "classe":"2A"}

modifica lo studente #2 a cui assegna una nuova residenza ed una nuova classe.

PATCH può aggiornare **UN SOLO RECORD** alla volta andando a sovrascrivere i campi passati come terzo parametro. Quindi non è necessario passare tutti i campi, ma SOLO quelli da aggiornare.

Se un campo non esiste viene automaticamente creato

A differenza di mongo consente di aggiornare anche l'ID, il che è estremamente pericoloso.

PUT /people/2 sostituisce l'intero record #2 con quello passato come **parametro**

POST /people {"nome":"pippo", "genere":"m", "classe":"2A"}
aggiunge in coda a **people** il record passato come terzo parametro

Nelle chiamate **POST** è buona norma non assegnare l'ID, nel qual caso json-server provvede automaticamente ad assegnare al nuovo record un ID casuale **univoco**, di tipo stringa lunga 4 caratteri. Restituisce dentro **response.data** l'intero record inserito, **comprensivo dell'ID** assegnato automaticamente. Non sembra possibile postare più record in una stessa chiamata.

Note

- Se in coda alla url permane un ? oppure anche ?& NON è un problema.
- Se uno dei campi di filtro ha un valore **undefined**, json-server semplicemente lo ignora.
- Il parametro **_limit=10** consente di limitare il numero di record restituiti.
- Il parametro **?q=testo** restituisce tutti i record che contengono quel testo, in un qualsiasi campo.
- **?titolo_like=testo** restituisce tutti i record in cui il campo 'titolo' contiene il testo indicato *Deprecato*

json annidati

Lo standard HTTP non definisce una sintassi ufficiale per rappresentare parametri annidati. Dipende da come il back end interpreta i dati

Chiamate GET

Per poter filtrare correttamente le proprietà annidate, **json-server** richiede esplicitamente la seguente notazione URL-encoded con il puntino

```
/people?location.country=italy
```

che in formato json diventa: `{"location.country":"italy"}`

Quindi la chiamata ajax dovrà *espressamente* avere il seguente formato:

```
inviaRichiesta( "GET", "/risorsa", {"location.country":"italy"} )
```

Viceversa, molte librerie javascript (come jQuery e axios) serializzano automaticamente i json annidati utilizzando la notazione con le parentesi quadre :

```
{location:{country:"italy"}} ) // che in json diventa  
location[country]=italy
```

formato che però json-server **non riconosce**

Volendo (*ma è abbastanza scomodo*) si potrebbe eseguire una conversione manuale delle quadre con il puntino, utilizzando ad esempio il metodo statico `$.param`

```
let result = $.param(json).replaceAll("%5B", ".").replaceAll("%5D", "")  
%5B e %5D sono le parentesi quadre. In questo modo si ottiene il classico formato url-encoded con il puntino; nome=pippo&location.city=London&location.street=Carnaby+Street
```

Viceversa in php ed express il discorso si ribalta: la notazione con le parentesi quadre è supportata nativamente mentre la notazione con il puntino NON viene interpretata.

Accesso ai dati

Notare che la precedente sintassi per il passaggio dei parametri annidati in formato url-encoded vale SOLO ed ESCLUSIVAMENTE in fase di passaggio dei parametri **URLencoded**

Viceversa non è utilizzabile per accedere ai campi di un generico json:

```
data.location.country  
data["location"]["country"]  
data["location.country"]
```

OK

OK

NOK

cerca una chiave che si chiama "location.country"

Chiamate PATCH

Nel caso delle chiamate PATCH, per aggiornare un sotto-campo occorre aggiornare e passare a **json-server** l'intero campo principale:

```
let location = person.location  
location.country = "italy"
```

```
let promise = inviaRichiesta("patch", "/pepole/"+people.id, {location} )
```

Campi vettoriali

json-server **NON** consente di ricercare singoli valori all'interno di campi vettoriali. Però riconosce singoli valori scalari posizionati all'interno di vettori lunghi 1.

Note sulle HTTP Request

Passaggio dell'ID come parte della risorsa.

In un server CRUD, per qualunque tipi di chiamata, l'ID viene di solito passato al server NON come parametro GET o POST, ma come parametro interno alla risorsa PRIMA di eventuale punto interrogativo

GET `http://mioServer/people/2`

Richieste HTTP GET

Quando non si passa l'ID, **eventuali altri parametri di filtro** devono essere accodati alla url in formato `URL_ENCODED`, cioè un elenco di coppie **Chiave=Valore** introdotte da un **?** e separate tra loro da una **&**

`http://www.miosito.it/pagina2.html?nome=mario&cognome=rossi&eta=16`

OR fra i parametri

Nei parametri URL-encoded lo standard HTTP consente di ripetere più volte lo stesso parametro, ad esempio: **GET** `/students?classe=1B&classe=2B`

ma **non definisce il significato logico** dei parametri ripetuti che spesso, a livello applicativo, vengono interpretati come una condizione di tipo OR.

I framework server-side, in presenza di parametri ripetuti, espongono generalmente i valori come un array, ad esempio: `classe = ['1B', '2B']`.

In questo caso è abitudine (ma non imposizione dello standard HTTP) terminare il nome del parametro con delle parentesi quadre: **GET** `/students?classe[]=1B&classe[]=2B`

In questo caso, il framework server-side interpreta e restituisce lo stesso vettore enumerativo di prima: `classe = ['1B', '2B']`.

E' vero anche il contrario: Molte librerie javascript come jQuery e axios convertono un json del tipo

```
{"classe" : ["1B", "2B "]}
```

nel seguente URL encoded:

```
classe[]=1B&classe[]=2B
```

spostando le quadre in coda alla chiave.

json-server, dalla versione 1.0, non supporta più questa modalità, supportata invece nella versione **0.17.4**

Richieste HTTP POST PATCH PUT

In caso di richiesta **HTTP POST** i parametri vengono passati al server **all'interno del body della HTTP REQUEST**, in modo non visibile. Maggiore riservatezza. Il formato utilizzato è di solito il formato JSON. La funzione `inviaRichiesta` provvede semplicemente a prendere il JSON ricevuto come terzo parametro, serializzarlo, e salvarlo all'interno del body della HTTP REQUEST

Modalità mista

Quando si esegue una chiamata **HTTP POST**, è anche possibile **aggiungere manualmente alcuni parametri GET in coda alla URL**.

In questo modo le informazioni passate come terzo parametro ad `inviaRichiesta()` verranno passate in modalità POST all'interno del body della HTTP REQUEST, mentre quelli direttamente concatenati alla URL verranno passati in modalità GET.

In caso invece di chiamata GET, eventuali parametri passati come terzo parametro a `inviaRichiesta()` sovrascrivono i parametri concatenati manualmente alla URL, per cui una modalità mista non è consentita.

Il controllo input[type=file]

Il controllo `<input type="file">` consente di selezionare uno o più files sul computer client tramite la tipica finestra "sfoglia". L'attributo `multiple` consente di selezionare contemporaneamente più files.

```
<input type="file" id="txtFile" accept=".pdf,.doc,.docx" multiple>
```

L'attributo `accept` serve a limitare i tipi di file selezionabili nel file picker del browser.

In corrispondenza dell'evento `onchange`, cioè dopo aver selezionato un file, `<input type="file">` restituisce all'interno dell'attributo dinamico `files` (accessibile solo da js) un vettore enumerativo di Oggetti File, dove File è un oggetto che eredita da Blob, quindi sostanzialmente un vettore di Blob.

Blob (Binary Large Object) è un contenitore generico in grado di contenere diverse tipologie di oggetti, con in aggiunta alcuni campi descrittivi.

Anche in assenza dell'attributo `multiple`, il controllo `<input type="file">` restituisce comunque **sempre** un vettore enumerativo di blob, contenente un solo file :

La proprietà `value` di input[type=file] restituisce una semplice stringa con il nome del files scelto

Nel caso di un File Object i campi presenti all'interno del Blob oltre all'immagine binaria sono i seguenti:

<code>"name"</code>	nome del file scelto dall'utente (<i>senza il path</i>).
<code>"size"</code>	dimensioni in bytes del file
<code>"type"</code>	contiene il MIME TYPE del file (es image/jpg)
<code>"lastModified"</code>	timestamp unix contenente la data dell'ultimo salvataggio del file
<code>"lastModifiedDate"</code>	serializzazione dell'informazione precedente

Il **contenuto binario** vero e proprio è nascosto all'interno del Blob ed è accessibile solo tramite **FileReader**

Preview dell'immagine

Poiché javascript non può accedere al file system, non è possibile fare una preview diretta dell'immagine. Per fare una preview occorre gestire la componente binaria presente all'interno del blob, la quale può essere convertita in base64 (`uri data`) per poi essere assegnata all'attributo `.src`

```
txtFile.addEventListener("change", async function () {
    let blob = txtFile.files[0];
    let base64Img = await base64Convert(blob).catch(function(err) {alert(err)})
    imgPreview.src = base64Img
})

function base64Convert(blob) {
    return new Promise(function(resolve, reject){
        let reader = new FileReader();
        reader.readAsDataURL(blob);
        reader.onload = function (event) {
            resolve(event.target.result); // event.target sarebbe reader
        };
        reader.onerror = function (error) {
            reject(error);
        };
    });
}
```

Come modificare l'aspetto grafico di un input type=file

Talvolta l'input type=file viene utilizzato in forma nascosta utilizzando una semplice label che, quando cliccata, **'forza'** il click sull'input type=file aprendo automaticamente la finestra di scelta del file:

```
<label for="newImg" class="btn btn-info">Add an image ...</label>
<input type="file" id="newImg" style="visibility:hidden;" >
```

Upload di una immagine

L'immagine scelta mediante `<input type="file">` può essere uploadata al server in due modi:

- Come semplice **stringa base64** trasmessa all'interno di un normale parametro **POST** di tipo string
- Come **immagine binaria** trasmessa tramite utilizzo dell'oggetto **FormData** utilizzando però un server che gestisca i form-data (cosa che json-server non fa)

Upload come stringa base64

L'immagine binaria restituita da `input[type=file]` può essere convertita in stringa base64 tramite utilizzo dell'oggetto `FileReader` e poi inserita come value di un qualsiasi parametro POST.

Il server, prima di salvarla su disco, dovrà eventualmente riconvertirla in binario.

Ricordare però che il file base64 pesa 4:3 in più rispetto al file binario.

Upload binario tramite l'oggetto FormData

Per trasmettere una immagine binaria al server, occorre caricarla all'interno di un oggetto denominato **FormData** aggiungendo al suo interno tutti i campi che si vogliono trasmettere al server.

```
let formData = new FormData()
formData.append('nome', txtNome.value);
formData.append('maggiorenne', chkMaggiorenne.checked);
formData.append("txtFile1", txtFiles.files[0]);
formData.append("myJSON", JSON.stringify(myJSON))
```

- **Tutti i parametri 'normali'** (cioè diversi dai Blob) vengono **automaticamente convertiti in stringa**
- All'interno di FormData è anche possibile passare un intero json, che però **deve essere serializzato** manualmente, altrimenti viene applicato il metodo `.toString()` che restituisce `Object Object`

Un oggetto FormData è un vettore ciclico, cioè un oggetto di tipo **key/value**. i contenuti rappresentano i **values** dell'oggetto, che vengono trasmessi così come sono, cioè come flussi binari senza controlli né conversioni. Nel momento in cui salviamo un Blob all'interno di un FormData, all'interno di value ritroviamo tutti i vari campi dell'oggetto File. Il contenuto binario sarà accessibile solo tramite `FileReader`.

Console.log() di un formData

`console.log(formData)` ha poco senso e non produce nessun effetto.

Per vedere il contenuto di una formData occorre utilizzare un ciclo del tipo :

```
for (let item of formData) {
    let name = item[0];    // chiave
    let value = item[1];   // contenuto
    console.log(name, value)
}
```



Caricamento di immagini multiple

Il metodo `formData.append()` consente di accodare più oggetti ad una stessa chiave che viene creata in corrispondenza del primo `append()`. In questi casi è bene assegnare al controllo `txtFile` un **nome vettoriale** del tipo `txtFiles[]`, in modo da consentire la lettura anche sui server più datati (esattamente nel caso di checkbox multipli aventi tutti lo stesso name).

Il client dovrà scorrere il vettore enumerativo restituito da `input[type=file]` e caricare ogni singolo File all'interno del FormData

```
let formData = new FormData()
for (let file of txtFiles.files)
  formData.append('txtFiles[]', file);
let rq = inviaRichiesta('POST', 'server/upload.php', formData);
```

Si potrebbero anche impostare chiavi differenti per ogni file, ma è più scomodo.

Utilizzo di un tag <form>

In alternativa è anche possibile inserire i controlli all'interno di una **form** e poi fare semplicemente

```
let formData = new FormData(form);
```

che equivale in tutto e per tutto al ciclo di caricamento manuale precedente. Vengono caricati anche i controlli semplici e NON SOLO quelli di tipo file.

Invio di una richiesta multipart

Nel caso di `$.ajax()` occorre inserire all'interno dei parametri le seguenti intestazioni HTTP:

```
options["headers"] ["Content-Type"]=false,
options["headers"] ["Process-Data"]=false,
options ["data"]= formData
```

Le prime due impostazioni fanno sì che la funzione `$.ajax()`, prima di trasmettere il file, non vada ad aggiungere una Content-Type header e non effettui una ulteriore serializzazione dei dati da inviare

Nel caso di `axios()` è sufficiente impostare l'intestazione Content-Type nel modo seguente

```
options["headers"] ["Content-Type"]='multipart/form-data';
options ["data"]= formData
```

Nel caso di `fetch()` occorre NON impostare il ContentType (cioè assegnarlo al valore false)

Form Submit (approccio SSR)

```
<form action="upload.php" method="post" enctype="multipart/form-data">
  Select file: <input type="file" name="txtFiles" id="txtFile" >
  <input type="submit" value="Upload">
</form>
```

In corrispondenza del submit, il file scelto viene trasmesso al server nel formato **name=value** come avviene normalmente per tutti i controlli della form (file binari e semplici controlli), per cui sostanzialmente non c'è nulla da scrivere. Il file viene inviato in forma binaria tramite utilizzo dell'oggetto FormData.

Il contenuto del file binario deve essere inviato all'interno del body della HTTP request e ovviamente non può essere concatenato alla URL. Per cui il metodo di trasmissione può essere soltanto **POST** e non GET

La parola **multipart** sta ad indicare che vengono trasmessi flussi binari puri con significato logico differente (immagini, video, object, semplici stringhe)

Nota: Se all'interno della **form** dovessero esserci più controlli di tipo `<input type="file">` l'oggetto FormData trasmesso al server presenterà più chiavi, una per ogni widget presente sulla form

Scambio dati in formato XML

Ricezione di uno stream xml relativo ad un elenco di studenti:

```
function aggiorna() {
  if (richiesta.readyState==4 && richiesta.status==200) {
    var tab = document.getElementById("gridStudenti");
    tab.innerHTML="";
    // Se si riceve un text/plain occorre parsificarlo
    var parser=new DOMParser();
    var xmlDoc=parser.parseFromString(richiesta.responseText, "text/xml");
    var root = xmlDoc.documentElement;

    // Se si riceve un albero già parsificato (content-type=application/xml)
    var root = richiesta.responseXML.documentElement;
    var table = document.getElementById("gridStudenti");
    table.innerHTML = "";
    var riga = document.createElement("tr");
    riga.innerHTML="<th>ID</th> <th>Nome</th> <th>Età</th> <th>Città</th>";
    table.appendChild(riga);

    for(var i=0; i<root.childNodes.length; i++){
      var record = root.childNodes[i]; // singolo studente
      var riga = document.createElement("tr");
      table.appendChild(riga);

      for (var j=0; j<record.childNodes.length; j++){
        var td;
        td = document.createElement("td");
        td.innerHTML = record.childNodes[j].textContent;
        riga.appendChild(td);
      }
    }
  }
}
```

Cenni sulla libreria sweetAlert2

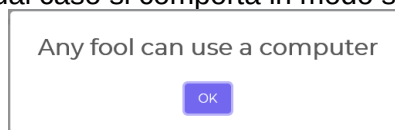
Consente di visualizzare delle finestre modali simili ad 'alert' e 'prompt' ma molto più friendly, con la possibilità di inserire all'interno molteplici contenuti html.

sweetAlert2 è *incompatibile* rispetto a sweetAlert 1. Viene distribuita come JS + CSS oppure **ALL** che contiene sia js che css. L'ultima versione a gennaio 2024 è la **11.10** disponibile al seguente cdn

```
https://cdnjs.cloudflare.com/ajax/libs/limonte-sweetalert2/11.3.7/sweetalert2.all.min.js
```

Per aprire una finestra sweetAlert2 si utilizza il metodo **Swal.fire()** che accetta come parametro:

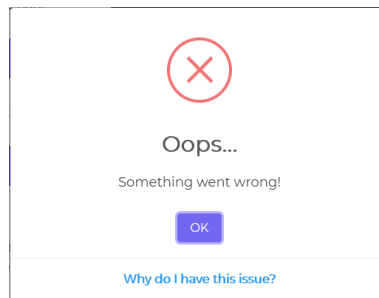
- Un json di opzioni
- Una semplice stringa, nel qual caso si comporta in modo simile ad una semplice alert()



Swal.fire() non restituisce una vera Promise javascript, ma un oggetto detto **thenable**, cioè che gestisce solo il then e non il catch, che in questi casi va in errore. Per cui il catch non è utilizzabile

Le opzioni di Swal.fire({ })

Di seguito un elenco delle principali opzioni utilizzabili all'interno di **Swal.fire()**



```
icon: 'success' 'error' 'warning' 'info' 'question'
iconColor: // colore dell'icona
title: 'Oops...' // titolo (accetta tag html di formattazione del testo)
text: 'Something went wrong!' // testo del messaggio
html: 'Something <b>wrong!</b>' // testo del messaggio formattato in html. Accetta anche un
// L'intero html della finestra può essere scritto qui dentro.
color: '#716add' // colore del testo
background: '#fff url(/images/trees.png)' // colore dello sfondo ed eventuale img
footer: '<a href="#">Why do I have this issue?</a>' // sotto il pulsante di chiusura

position: 'top-end' // posizione di apertura della finestra
width: 600 // larghezza della finestra including paddings (box-sizing: border-box)
padding: '3em' // 3 volte rispetto al valore del font-size

showConfirmButton: true // visualizzazione del Confirm Button (OK)
confirmButtonColor: #aaa // colore di sfondo Confirm Button
confirmButtonText: '<i class="fa fa-thumbs-up"></i> Great!' // dito su
```

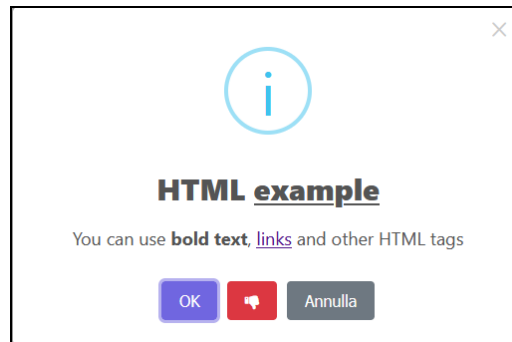
Ajax

```

showDenyButton: false // visualizzazione del Deny Button (NO)
denyButtonColor: '#aaa' // colore di sfondo del Deny Button
denyButtonText: '<i class="fa fa-thumbs-down"></i>' // dito giù

showCancelButton: false // visualizzazione del Cancel Button (CANCEL)
cancelButtonColor: '#aaa' // colore di sfondo del Cancel Button
cancelButtonText: 'Annulla'

```



```

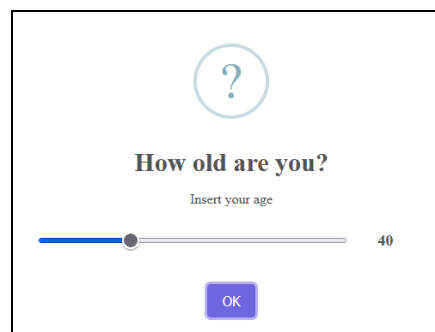
reverseButtons: false // se true inverte la posizione dei tre Default Buttons precedenti
showCloseButton: true // pulsante di chiusura in alto a destra
timer: 1500 // timer di autochiusura della finestra (con tutti i pulsanti a false)
focusConfirm: true, // se true il focus sarà sul pulsante di conferma, altrimenti sul primo elemento della finestra

```

```

input: 'range' // un input type per l'inserimento di un valore
inputLabel: 'Insert your age' // label associata al precedente tag input
inputPlaceholder
inputAttributes: {min: 20, max: 90, step: 1} // attributi del tag input
inputValue: 40 // value dal tag input che verrà iniettato al metodo .then()

```



```

imageUrl: 'https://unsplash.it/400/200', // visualizza una immagine
imageWidth: 400,
imageHeight: 200,
showClass: { popup: 'animate__animated animate__fadeInDown' } // effetto in apertura
hideClass: { popup: 'animate__animated animate__fadeOutUp' } // effetto in chiusura

```

Lettura del risultato

In tutti i casi il metodo `fire()` restituisce SEMPRE una Promise alla cui funzione di callback viene iniettato un oggetto **result** contenente campi indicati di seguito:

```
.then(function(result) {
    console.log(result)
})

result.isConfirmed           // true se è stato premuto il Confirm Button
result.isDenied             // true se è stato premuto il Deny Button.
result.isDismissed          // true se è stato premuto il Cancel Button.
result.dismiss // consente di approfondire la causa del cancel. I valori possibili sono:
    Swal.DismissReason.cancel
    Swal.DismissReason.close
    Swal.DismissReason.esc
    Swal.DismissReason.timer
    Swal.DismissReason.backdrop

value:                      // valore dell'eventuale input type presente.
                             Se non ci sono tag di input sarà true nel caso del Confirm Button e false negli altri due casi
```

await

L'`await`, come sempre, restituisce direttamente l'oggetto **result** iniettato al `then`

```
let result = await Swal.fire({ })
console.log(result)
```

Elementi di input multipli

Nel caso di pagine html contenenti più elementi di input, tutti i vari risultati possono essere ritornati alla funzione di callback nel modo seguente:

```
Swal.fire({
  title: 'Multiple inputs',
  html:
    '<p> nome </p>' +
    '<input id="txtNome" class="swal2-input">' +
    '<p> età </p>' +
    '<input id="txtEta" class="swal2-input">',
  preConfirm: () => {
    return [
      document.getElementById('txtNome').value,
      document.getElementById('txtEta').value
    ]
  }
}).then(function(result) {
  console.log(result.value)
})
```

In questo caso **result.value** sarà un vettore enumerativo contenente tutte le voci inserite secondo l'ordine utilizzato all'interno del **preConfirm**.

In alternativa, all'interno del **then**, è anche possibile accedere in modo diretto a qualsiasi elemento della `sweetAlert` mediante il puntatore `js` all'**ID**:

```
let nome = txtNome.value
```

Cenni sulla libreria chart.js (versione 3)

chartjs.org è una libreria JavaScript, intrinsecamente **responsive**, open-source, molto flessibile, che permette di creare rapidamente dei grafici su un canvas HTML5 (area di disegno). Si aspetta come parametro il canvas su cui tracciare il grafico ed un json contenente sia il tipo di grafico sia i dati da visualizzare, memorizzati sotto forma di vettori enumerativi paralleli.

Per l'utilizzo in una applicazione javascript sono disponibili diversi **CDN**

<https://cdn.jsdelivr.net/npm/chart.js> // Ultima versione

<https://cdnjs.cloudflare.com/ajax/libs/Chart.js/4.2.0/chart.min.js>

Notare che la versione 4.2.0 è esposta come modulo **ESM** (ES6 Modules)

E' però disponibile anche la classica versione **UMD** (Universal Module Definition)

La versione `bundle.js` congloba anche `moment.js` per la gestione delle date

I tipi di grafici supportati

```
'doughnut' -> ciambella
'pie'       -> torta
'bar'       -> diagramma a barre verticali
'line'      -> diagramma per punti
'radar'     -> diagramma a ragnatela
```

Esistono poi altri grafici a base tridimensionale (es bubble, scatter, etc.) che si aspettano come values un vettore enumerativo di JSON costituiti da tre campi:

```
{  x: number,    // coordinata X
  y: number,    // coordinata Y
  r: number     // Bubble radius in pixels. }
```

Le impostazioni principali sono più o meno le stesse per tutti i tipi di grafico. Per ogni tipo ci sono comunque delle impostazioni specifiche come ad esempio **fill:true** per il `type:line` che ricolore l'area sottostante con il colore definito all'interno di `backgroundColor`

Impostazione delle chartOptions

```
let chartOptions = {
  type: 'bar',
  data: {
    labels: ['pippo', 'pluto', 'minnie'], // x axis keys
    datasets: [{
      label: 'risultati scolastici', // label of dataset (*)
      data: [12, 19, 32], // values
      fill: true, // solo per type=line
      backgroundColor: [ // colore delle singole barre
        'rgba(255, 99, 132, 0.2)',
        'rgba(54, 162, 235, 0.2)',
        'rgba(255, 206, 86, 0.2)'
      ],
      borderColor: [ // colore dei bordi delle barre
        'rgba(255, 99, 132, 1)',
        'rgba(54, 162, 235, 1)',
        'rgba(255, 206, 86, 1)'
      ],
      borderWidth: 1 /* default 2 */
    }]
  },
  options: { }
```

Note

Nel caso si voglia utilizzare lo stesso colore per tutti i record, al posto di un vettore si può assegnare una semplice stringa con il colore desiderato

(*) Per ogni grafico è possibile rappresentare contemporaneamente **più datasets** (es risultati 2020, risultati 2021, risultati 2022, etc) ognuno con i propri dati ed i propri colori. Il campo label interno ad un singolo dataset, rappresenta il titolo del singolo dataset. .

Il **titolo principale** del grafico va messo invece all'interno della chiave **options** alla voce **plugins**.

```
options: {
  scales: {
    y: {
      suggestedMax: 40,
      suggestedMin: -10,    // oppure
      beginAtZero: true
    }
  },
  plugins: {
    // titolo principale del grafico
    title: {
      display: false, // default
      text: 'Main Title'
    }
    // per visualizzare il legend dei diagrammi multi-bar o pie
    legend: {
      display: true,    // default
      position: 'top',  // Il testo è dentro ogni singolo dataset
    },
  },
  animation: {
    onComplete: function() {alert("ok")}
  }
}
```

scales fa sì che venga disegnato l'asse verticale fra i valori MIN e MAX indicati.

Utile nel caso del diagramma a barre in cui, per default, come valore minimo di partenza delle barre viene automaticamente impostato il dato con valore minimo.

Per cui l'oggetto con valore minimo avrebbe una barra con altezza zero.

animation Per default il grafico viene visualizzato con una certa animazione in ingresso (con tempo fisso di 1000 msec) che può essere disabilitata impostando

```
chart.options.animation = false;
```

Istanza del grafico

```
<div id="canvasContainer">
  <canvas> </canvas>
</div>
```

Il canvas html deve necessariamente essere inserito all'interno di un proprio tag DIV avente WIDTH e HEIGHT ben definiti.

```
let canvas = document.getElementsByTagName('canvas')[0] // puntatore js !!
let chart = new Chart(canvas, chartOptions);
```

Notare che il disegno del grafico lanciato tramite new Chart() è **asincrono**.

Update del grafico

L'istanza del grafico può essere fatta una sola volta all'inizio.

Per aggiornare il grafico con i nuovi dati ricevuti dal server, occorre aggiornare i due vettore `labels[]` e `data[]` e poi richiamare il metodo

```
chart.update()
```

Se invece occorre re-istanziare l'intera **chart** da associare al medesimo canvas, prima di istanziare il Chart, occorre **necessariamente** rimuovere il chart precedente con un destroy

```
if(chart)
    chart.destroy()    // oppure chart.remove()
chart = new Chart(canvas, chartOptions)
```

A questo scopo la variabile **chart** dovrà necessariamente essere dichiarata globale.

Responsive

Per rendere il grafico responsive occorre aggiungere le seguenti **options**:

```
responsive:true,
aspectRatio:true
maintainAspectRatio: false,
```

Dopo di che occorre assegnare al contenitore una larghezza **relativa** in funzione della larghezza della pagina. L'altezza invece può essere assegnata a piacimento.

Impostazione dinamica delle chartOptions

Tutte le chartOptions, oltre che essere definite staticamente in fase di strutturazione del grafico, possono poi anche essere impostate / modificate dinamicamente sulle singole istanze:

```
chart.options.animation.duration = 0;
```

Cenni sulla libreria crypto-js

```
<script src = "https://cdnjs.cloudflare.com/ajax/libs/crypto-js/4.0.0/crypto-
js.min.js"> </script>
```

Consente il calcolo di impronte md5, sha256, aes, base64.

Esempio :

```
let pass = CryptoJS.MD5(_txtPassword.val()).toString();
```